

Day 15: Encoder vs Decoder (BERT in detail)

1.1 What is a Transformer?

The Transformer was introduced by Google Brain in the paper "Attention is All You Need" (Vaswani et al., 2017). It completely replaced RNNs and LSTMs as the dominant architecture for NLP. The core idea is Self-Attention — every token in a sequence can directly attend to every other token, regardless of distance.

Original architecture: The 2017 paper introduced a full Encoder-Decoder structure. The Encoder reads the input, the Decoder generates the output. Over time, researchers discovered you do not always need both halves — and this gave rise to three distinct model families.

1.2 The Three Transformer Families

Family	Which Part Used	Attention Style	What It Does	Real Examples
Encoder-Only	Encoder stack only	Bidirectional — each token sees ALL other tokens in both directions	Understands and represents text. Used for classification, NER, QA, semantic search.	BERT, RoBERTa, ALBERT, DistilBERT, ELECTRA, DeBERTa
Decoder-Only	Decoder stack only	Causal / Autoregressive — each token only sees tokens to its LEFT	Generates text token by token. Used for chatbots, code generation, text completion.	GPT-2, GPT-3, GPT-4, LLaMA, Mistral, Gemini
Encoder-Decoder	Both Encoder + Decoder	Encoder: Bidirectional. Decoder: Causal	Transforms one sequence into another. Used for translation, summarisation, question answering.	T5, BART, mT5, PEGASUS

--	--	--	--	--

1.3 Encoder — Deep Explanation

The Encoder reads an entire input sequence and produces a rich contextual representation of it. Because every token can attend to every other token simultaneously (bidirectional attention), the Encoder understands the full meaning of each word in context — from both the left side and the right side of the sentence.

How the Encoder works — step by step

- **Step 1 — Tokenisation:** The input sentence is split into subword tokens (e.g., "running" → "run", "##ning").
- **Step 2 — Embedding:** Each token is converted into a vector (a list of numbers). Position information is added so the model knows token order.
- **Step 3 — Self-Attention:** Each token creates three vectors — Query (Q), Key (K), Value (V). Attention score = $\text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$. Every token attends to every other token, producing a context-enriched representation.
- **Step 4 — Feed-Forward Layer:** Each token's representation passes through a small neural network independently.
- **Step 5 — Layer Stacking:** Steps 3–4 repeat N times (e.g., 12 layers in BERT-Base). Each layer refines the representation further.

Key Characteristic of the Encoder

Bidirectional attention — the word 'bank' in the sentence 'I went to the bank to deposit money' attends to both 'deposit' (on its right) and 'went' (on its left) simultaneously.

This is why Encoder models understand context deeply.

But: Encoder models CANNOT generate new text natively because they see the full sequence at once — there is no notion of 'what comes next'.

Examples of Encoder-Only Models

- BERT — the original, covered in full detail in Part 2
- RoBERTa — improved BERT training recipe
- ALBERT — lightweight BERT with parameter sharing
- DistilBERT — compressed BERT via knowledge distillation
- ELECTRA — efficient BERT via replaced token detection
- DeBERTa — BERT with disentangled attention (content + position separated)

1.4 Decoder — Deep Explanation

The Decoder generates text one token at a time, left to right. It uses Causal (Masked) Self-Attention, which means when predicting the next token, the model can only see tokens that have already been generated — it cannot look ahead.

How the Decoder works — step by step

- **Step 1 — Input so far:** The decoder receives the tokens generated so far (or a prompt). It processes the sequence with left-to-right attention only.
- **Step 2 — Masked Self-Attention:** A causal mask blocks all future positions. Token at position i can only attend to positions 0 through i . This ensures the model does not cheat by looking ahead.
- **Step 3 — Feed-Forward Layer:** Same as the encoder — a small MLP processes each token representation.
- **Step 4 — Language Model Head:** The final representation of the last token is projected through a linear layer and softmax over the full vocabulary. The model picks the next token (greedy, top-k, nucleus sampling, etc.).
- **Step 5 — Repeat (Autoregressive Loop):** The predicted token is appended to the sequence. The whole process repeats until an end token or max length is reached.

Key Characteristic of the Decoder

Causal attention — each token only sees what came before it.

This makes Decoders perfect for generation tasks: text completion, chatbots, code generation.

But: because they only see left context, Decoders are weaker at understanding tasks like classification or NER compared to Encoders.

Examples of Decoder-Only Models

- GPT-2 — OpenAI's 2019 text generation model (1.5B parameters)
- GPT-3 — 175B parameter model, the basis of early ChatGPT
- GPT-4 — OpenAI's current flagship (multimodal, reasoning-capable)
- LLaMA / LLaMA-2 / LLaMA-3 — Meta's open-source decoder family
- Mistral 7B — efficient open decoder, strong at instruction following
- Gemini — Google DeepMind's multimodal decoder family

1.5 Encoder vs Decoder — Side-by-Side

Aspect	Encoder (e.g., BERT)	Decoder (e.g., GPT)
Attention Direction	Bidirectional — sees all tokens	Causal — sees only left context
Primary Purpose	Understand / represent text	Generate text
Training Task	Masked Language Modeling (MLM)	Next Token Prediction
Output	Contextualised token representations	Next token probability distribution

Good for	Classification, NER, QA, search	Chatbots, code gen, summarisation
Cannot do natively	Text generation	Token-level understanding tasks well

2.1 What is BERT?

BERT was published by Google AI in October 2018 by Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. It was the first large-scale pre-trained Encoder-only Transformer. BERT set new state-of-the-art on 11 NLP benchmarks simultaneously when released — a historic moment in NLP.

Full form: Bidirectional Encoder Representations from Transformers

Core idea: Pre-train a deep bidirectional Transformer on massive unlabelled text, then fine-tune it on any NLP task with minimal task-specific modification.

2.2 Why was BERT Revolutionary?

- **Before BERT:** Models like ELMo used shallow bidirectionality — two separate unidirectional LSTMs (one left-to-right, one right-to-left) whose representations were concatenated. This is not truly bidirectional.
- **Before BERT:** GPT-1 (2018) used a deep Transformer but was strictly left-to-right (unidirectional), limiting its ability to understand context.
- **BERT's breakthrough:** Used Masked Language Modeling (MLM) to train a truly bidirectional Transformer — every layer attends to the full context, both left and right simultaneously.

2.3 BERT Architecture

BERT uses only the Encoder stack. It is essentially N stacked Transformer Encoder blocks.

Parameter	BERT-Base	BERT-Large
Encoder Layers (L)	12	24
Hidden Size (H)	768	1024
Attention Heads (A)	12	16
Feed-Forward Size	3072	4096
Total Parameters	110 Million	340 Million
Max Sequence Length	512 tokens	512 tokens
Training Hardware	16 TPU v3 chips, 4 days	64 TPU v3 chips, 4 days

2.4 BERT Input Representation

BERT's input is constructed by summing three separate embeddings for each token. These three embeddings are learned during pre-training.

Embedding Type	What It Encodes	Detail
Token Embedding	The identity of the word/subword token	Uses WordPiece tokenisation. Vocabulary of 30,522 tokens. Unknown words are split into known subwords.
Segment Embedding	Which sentence the token belongs to	Embedding A for the first sentence, Embedding B for the second. Tells BERT where Sentence A ends and Sentence B begins.
Position Embedding	The position of the token in the sequence	Learned (not sinusoidal like the original Transformer). Values 0 to 511 covering max 512 positions.

Final Input Formula
Final Input Vector = Token Embedding + Segment Embedding + Position Embedding
All three vectors have the same dimension (768 for Base, 1024 for Large) and are added element-wise.

2.5 Special Tokens

Token	Full Name	Where Used	Purpose
[CLS]	Classification Token	Prepended to EVERY input sequence	Its final hidden state (768-dim vector) is used as the aggregate sequence representation for classification tasks.
[SEP]	Separator Token	Between Sentence A and Sentence B, and at the end	Tells BERT where one sentence ends and the next begins.
[MASK]	Mask Token	During MLM pre-training only	Replaces selected tokens that BERT must predict. Never seen at fine-tuning time.
[PAD]	Padding Token	When batching sequences of different lengths	Pads shorter sequences so all sequences in a batch are the same length.
[UNK]	Unknown Token	Any out-of-vocabulary character or sequence	Represents tokens that cannot be formed from known subwords.

2.6 Pre-training — Task 1: Masked Language Modeling (MLM)

MLM is the core innovation that enables truly bidirectional learning. BERT randomly hides some tokens and asks the model to predict them using all surrounding context.

Masking Strategy

- 15% of input tokens are randomly selected for masking in each training example
- Of those 15% selected tokens:
 - 80% are replaced with the special [MASK] token
 - 10% are replaced with a random word from the vocabulary
 - 10% are kept unchanged (original word, no replacement)
- BERT must predict the original token at every masked position
- The 10% random + 10% unchanged trick prevents the model from learning to only look at [MASK] positions — it must treat every token as potentially needing prediction

Why Bidirectionality Requires MLM
If you train a model to predict the NEXT token (like GPT), it can only look left — otherwise it would just read the answer.
MLM hides tokens inside the sequence. To predict a hidden token, the model must look at BOTH the left context and the right context.
This is what forces BERT to learn genuinely bidirectional representations at every layer.

Concrete Example

Original: "The cat sat on the mat"
After masking (80% case): "The cat [MASK] on the mat"
BERT must predict 'sat' using context from both 'The cat' (left) and 'on the mat' (right).

2.7 Pre-training — Task 2: Next Sentence Prediction (NSP)

NSP trains BERT to understand the relationship between two sentences. This was designed to help with tasks like Question Answering and Natural Language Inference.

How NSP Works

- Input format: [CLS] Sentence A [SEP] Sentence B [SEP]
- 50% of the time — Sentence B is the actual next sentence after A in the original corpus (label: IsNext)
- 50% of the time — Sentence B is a random sentence from a different document (label: NotNext)
- BERT uses the [CLS] token's output representation to predict IsNext or NotNext via a binary classifier

Note on NSP

Later research (RoBERTa, ALBERT) found that NSP as originally implemented in BERT may actually hurt performance.

The task was too easy — random sentences from different documents are obviously unrelated.

Most BERT variants either remove NSP entirely (RoBERTa) or replace it with a harder version (ALBERT uses Sentence Order Prediction).

2.8 BERT Fine-tuning

BERT follows a pre-train once, fine-tune everywhere paradigm. After pre-training (done by Google, once, expensively), you add a small task-specific output head and train on your labelled data for just a few epochs.

Task Type	What You Add on Top of BERT	Which BERT Output Used	Example Tasks
Single Sentence Classification	One linear layer + softmax	[CLS] token's final hidden state	Sentiment analysis, spam detection, topic classification
Sentence Pair Classification	One linear layer + softmax	[CLS] token's final hidden state	Natural Language Inference (entailment), paraphrase detection
Token Classification	One linear layer per token + softmax	Each token's final hidden state	Named Entity Recognition (NER), Part-of-Speech tagging
Question Answering (Extractive)	Two linear layers (start + end span)	Each token's final hidden state	SQuAD — find the answer span within a passage
Semantic Similarity	Linear layer + cosine or dot-product head	[CLS] or mean of all token states	Sentence similarity, information retrieval, semantic search

2.9 BERT Pre-training Data

- BooksCorpus — 800 million words from unpublished books
- English Wikipedia — 2,500 million words (text only, no tables/headers)
- Total corpus: approximately 3.3 billion words
- Vocabulary: 30,522 WordPiece subword tokens
- Max sequence length: 512 tokens
- Training: 1 million steps, batch size 256 sequences

2.10 Known Limitations of BERT

- **[MASK] mismatch:** The [MASK] token appears during pre-training but never during fine-

tuning. This creates a distribution gap between the two phases.

- **NSP weakness:** The NSP task was found to be too easy and sometimes adds noise rather than helping downstream tasks.
- **Quadratic attention cost:** Self-attention scales as $O(n^2)$ with sequence length. BERT becomes very slow and memory-intensive for sequences longer than 512 tokens.
- **Cannot generate text:** BERT is not autoregressive — it cannot natively produce text outputs. It requires a separate decoder or generation method.
- **Undertrained:** RoBERTa later showed that BERT was significantly undertrained. The same architecture, trained longer and with more data, performs substantially better.

3.0 Overview: Why Do Variants Exist?

Every BERT variant below addresses one or more of BERT's limitations. Understanding why each variant was created is more important than memorising its name.

Model	Organisation	Year	Core Problem It Solves
RoBERTa	Facebook AI Research	2019	BERT was undertrained — fix the training recipe
ALBERT	Google Research	2019	BERT is too large — reduce parameters via sharing
DistilBERT	HuggingFace	2019	BERT is too slow/large for production — compress it
SpanBERT	Facebook AI	2019	Token-level masking is weak for span tasks — mask spans
XLNet	CMU + Google Brain	2019	BERT's [MASK] mismatch + unidirectional GPT limitation — combine both worlds
ELECTRA	Google Brain	2020	BERT's MLM is inefficient — use all tokens not just 15%
DeBERTa	Microsoft Research	2020	BERT conflates content and position in attention — disentangle them
ERNIE	Baidu	2019	BERT masks random tokens — inject real-world knowledge by masking entities
SciBERT	Allen AI	2019	BERT vocabulary is not suitable for scientific text
BioBERT	DMIS Lab	2019	BERT vocabulary is not suitable for biomedical text
FinBERT	Prosus AI	2019	BERT vocabulary is not suitable for financial text
ClinicalBERT	MIT CSAIL	2019	BERT vocabulary is not suitable for clinical notes
mBERT	Google	2018	BERT only works on English — extend to 104 languages
XLM-RoBERTa	Facebook AI	2019	mBERT is weak — apply RoBERTa recipe to multilingual setting

3.1 RoBERTa

RoBERTa — Robustly Optimized BERT Approach

— Facebook AI

Research (2019)

"BERT was significantly undertrained. The same architecture, trained correctly, is much more powerful."

- **Key Finding:** BERT was undertrained. RoBERTa did not change the architecture at all — it only changed the training procedure.
- **Removes NSP entirely:** Facebook found that NSP does not help and sometimes hurts. RoBERTa trains on single long sequences packed into 512 tokens, with no second-sentence prediction task.
- **Dynamic Masking:** BERT uses static masking — the same tokens are always masked for a given training example across all epochs. RoBERTa generates a new mask every time the same sequence appears during training. This means the model sees far more diverse masking patterns.
- **10x More Data:** BERT trained on 16GB of text. RoBERTa used 160GB: BooksCorpus + English Wikipedia + CC-News + OpenWebText + Stories.
- **Larger Batch Sizes:** BERT used batch size 256. RoBERTa found that training with batch sizes of 8,000 or even 32,000 sequences is significantly more stable and effective.
- **Larger BPE Vocabulary:** RoBERTa uses byte-level BPE with a vocabulary of 50,265 tokens (vs BERT's WordPiece with 30,522 tokens). This covers a wider range of rare words and subword patterns.
- **Results:** RoBERTa outperforms BERT-Large on GLUE, SQuAD 1.1, SQuAD 2.0, and RACE benchmarks — without any change to the model architecture.
- **Key lesson:** Training methodology matters as much as architecture design.

3.2 ALBERT

ALBERT — A Lite BERT

— Google Research (2019)

"Can we achieve BERT-level performance with a fraction of the parameters?"

- **Problem ALBERT solves:** BERT-Large has 340M parameters. Pre-training and deploying it is extremely expensive. ALBERT reduces this dramatically through two architectural changes.
- **Innovation 1 — Factorized Embedding Parameterization:** In BERT, the vocabulary embedding size equals the hidden size (both 768). This wastes parameters because vocabulary embeddings need not be as expressive as hidden representations. ALBERT separates these: vocabulary embedding dimension = 128, hidden dimension = 768. This reduces embedding parameters from $V \times H$ ($30522 \times 768 = 23.4\text{M}$) to $V \times E + E \times H$ ($30522 \times 128 + 128 \times 768 = 3.9\text{M} + 0.1\text{M} = \sim 4\text{M}$). This is an 80% reduction in embedding parameters.
- **Innovation 2 — Cross-Layer Parameter Sharing:** BERT has 12 separate Transformer layers, each with their own weights. ALBERT uses the SAME weights for all 12 (or 24) layers. Instead of 12 independent sets of attention + FFN parameters, there is exactly 1 set, reused at every layer. This is the dominant source of parameter reduction.
- **Replaces NSP with SOP:** ALBERT replaces Next Sentence Prediction with Sentence Order Prediction (SOP). In SOP, both sentences always come from the same document — the model must predict if they are in the correct order or swapped. This is a harder, more meaningful task that captures coherence, not just topicality.
- **ALBERT-xxlarge:** Has only 235M parameters (vs BERT-Large's 340M) yet outperforms BERT-Large on GLUE and SQuAD. It achieves this through a very deep

3.3 DistilBERT

DistilBERT — Distilled BERT — HuggingFace (2019)

"Make BERT small enough and fast enough for production use and edge deployment."

- **Knowledge Distillation concept:** A large pre-trained Teacher model (BERT) teaches a smaller Student model (DistilBERT). The student does not learn from labelled data directly — it learns to reproduce the teacher's behaviour.
- **Architecture reduction:** DistilBERT uses 6 Transformer layers instead of BERT's 12. Token type embeddings and the pooler layer are removed. Parameters reduced from 110M to 66M (40% smaller).
- **Initialisation trick:** DistilBERT is not trained from scratch. The 6 student layers are initialised by taking every other layer from BERT-Base (layers 0, 2, 4, 6, 8, 10). This gives the student a warm start from the teacher's representations.
- **Three training losses:** 1) Distillation Loss — soft cross-entropy between student and teacher probability distributions over vocabulary. Soft probabilities carry more information than one-hot labels (e.g., even the wrong predictions reveal what the teacher thinks is similar). 2) Cosine Embedding Loss — the student's hidden states are trained to align directionally with the teacher's hidden states. 3) MLM Loss — standard masked language model loss on ground truth tokens.
- **Results:** 66M parameters (40% smaller), 60% faster inference, 97% of BERT-Base performance on GLUE benchmark.
- **Best use case:** Real-time NLP APIs, mobile/edge deployment, high-throughput production systems where accuracy vs latency trade-off matters.

3.4 SpanBERT

SpanBERT — Span-level BERT — Facebook AI (2019)

"Random individual token masking is not enough — many NLP tasks require reasoning over spans of text."

- **Core innovation — Span Masking:** Instead of masking random individual tokens, SpanBERT selects and masks contiguous spans of text. The span length is sampled from a geometric distribution (mean 3.8 tokens). This forces the model to learn about phrases, entities, and multi-word expressions, not just individual tokens.
- **Span Boundary Objective (SBO):** This is the key new training task. To predict any token inside a masked span, the model is only allowed to use the two boundary tokens — the tokens immediately to the left and right of the masked span. It cannot use the tokens adjacent to the one it is predicting (which would make it too easy). This forces the model to encode rich span-level information into the boundary representations.
- **Removes NSP:** SpanBERT trains on single contiguous segments (no paired sentences). NSP was found to not benefit span-level reasoning tasks.
- **Results:** Significantly outperforms BERT on all span-prediction tasks: SQuAD 1.1, SQuAD 2.0, coreference resolution (OntoNotes), and relation extraction (TACRED).

3.7 DeBERTa

DeBERTa — Decoding-enhanced BERT with Disentangled Attention — Microsoft Research (2020)

"BERT mixes content and position information in the same attention vector. Separate them for richer representations."

- **The limitation it addresses:** In BERT, each token's representation encodes both its content (what word it is) and its position (where it is). During attention, these are mixed together. DeBERTa argues these should be treated separately because their interactions (content-to-content, content-to-position, position-to-content) carry different kinds of information.
- **Disentangled Attention Mechanism:** Each token is represented by TWO separate vectors — one for content (what the word is) and one for position (where it is). Attention is then computed as a sum of four separate attention scores: content-to-content (standard attention), content-to-position (how word meaning relates to relative positions), position-to-content (how position relates to word meanings around it), and position-to-position (excluded as less useful). This gives a much richer attention mechanism.
- **Relative Position Encoding:** DeBERTa uses relative (not absolute) position embeddings. Instead of embedding 'I am at position 5', the model encodes 'I am 3 positions to the left of this token'. This is more generalisable across different sequence lengths and positions.
- **Enhanced Mask Decoder:** Absolute position embeddings are only introduced in the final few layers before the token prediction head, not in every layer. This means the model uses relative positions throughout most of its processing and only injects absolute positions when needed for prediction.
- **DeBERTa v2 and v3:** v2 adds a larger vocabulary (128K tokens) and virtual adversarial training (adding small noise to embeddings for regularisation). v3 combines disentangled attention with ELECTRA-style pre-training (Replaced Token Detection), giving a very powerful and efficient encoder.
- **Results:** DeBERTa-v3-large achieves state-of-the-art on SuperGLUE, SQuAD, and many other benchmarks. It is currently one of the strongest off-the-shelf encoders for fine-tuning tasks.

3.9 Domain-Specific BERT Variants

The following models take BERT (or RoBERTa) and continue pre-training on large domain-specific text corpora. The architecture is unchanged — only the training data differs. This is called Domain-Adaptive Pre-training (DAPT).

Why this works: General BERT vocabulary and representations are poor at domain-specific jargon. A word like 'EGFR' (a gene) or 'EBITDA' (a financial metric) appears rarely or never in Wikipedia/BooksCorpus. Domain-specific pre-training teaches BERT the vocabulary, abbreviations, and writing patterns of a target domain.

SciBERT — Allen Institute for AI (AllenAI) (2019)

BERT for scientific literature — research papers, academic writing, technical terminology.

- **Training data:** 1.14 million scientific papers from Semantic Scholar: 18% computer science, 82% biomedical sciences. 3.17 billion tokens total.
- **Custom vocabulary — SCIVOCAB:** A new vocabulary of 30K tokens built from scientific text using WordPiece. Scientific papers use very different vocabulary from general English — SCIVOCAB covers scientific terms that BERT's general vocabulary would split into meaningless fragments.
- **Two versions:** scivocab-cased (custom vocab, case-sensitive) and basevocab-cased

(BERT vocab, case-sensitive). Scivocab version is stronger for scientific tasks.

- **Best for:** Scientific NER (gene/protein/chemical names), paper classification, citation recommendation, relation extraction in research papers.

BioBERT — DMIS Lab, Korea University (2019)

BERT for biomedical text — PubMed abstracts, medical research, clinical trials.

- **Training data:** Initialised from BERT-Base, then further pre-trained on: PubMed abstracts (4.5 billion words from 30 million papers) and PubMed Central (PMC) full-text articles (13.5 billion words). Combined: 18 billion biomedical tokens.
- **Why important:** Biomedical text is dense with highly specialised terminology — drug names, gene symbols, disease names, chemical compounds. Standard BERT tokenises 'mTOR' as 'm', '##TO', '##R' — three meaningless fragments. BioBERT's pre-training teaches it that 'mTOR' is a meaningful entity (a protein kinase).
- **Results:** Significantly outperforms BERT on biomedical NER (BC5CDR, NCBI-disease), biomedical relation extraction, and biomedical QA (BioASQ).
- **Versions:** BioBERT v1.0, v1.1 (more PubMed data), v1.2 (full PubMed text). Also Bio+ClinicalBERT (further fine-tuned on clinical notes on top of BioBERT).

FinBERT — Prosus AI / Hugging Face Community (2019)

BERT for financial text — earnings calls, financial news, analyst reports.

- **Training data:** Initialised from BERT-Base, continued pre-training on: Reuters TRC2 news corpus (financial news), Financial PhraseBank (manually annotated financial sentences), and analyst report corpora.
- **Fine-tuning:** Specifically fine-tuned for financial sentiment analysis — classifying financial text as positive, neutral, or negative in sentiment.
- **Why general BERT fails:** General BERT does not understand that 'the company beat consensus estimates by 15%' is positive, or that 'increasing provisioning for credit losses' signals financial stress. These concepts require domain-specific training.
- **Use cases:** Earnings call sentiment scoring, financial news tone classification, risk factor analysis in 10-K filings, trading signal generation from news.

3.10 Multilingual BERT Variants

mBERT — Multilingual BERT — Google (2018)

One BERT model pre-trained on 104 languages simultaneously.

- **Training data:** Wikipedia text from 104 languages. Languages are sampled with a smoothed distribution to give more weight to lower-resource languages (otherwise English would dominate).
- **Shared vocabulary:** 119,547 WordPiece tokens shared across all 104 languages. The model must represent all languages in a common embedding space.
- **Zero-shot cross-lingual transfer:** Remarkably, mBERT develops cross-lingual representations without any cross-lingual supervision. Fine-tune on English NER → evaluate on German NER, and it still performs reasonably well. This emergent behaviour suggests the model aligns representations across languages.
- **Limitation:** High-resource languages (English, German, French) get much more training data than low-resource ones. Performance on low-resource languages is significantly weaker. Also, languages using non-Latin scripts (Arabic, Chinese, Hindi) share fewer subwords with Latin-script languages.
- **Pre-training task:** Standard MLM only, applied to each language's Wikipedia text. No cross-lingual alignment objective.

XLNet — Facebook AI Research (2019)

Multilingual RoBERTa — far stronger than mBERT, the go-to multilingual encoder.

- **Training data:** 2.5 TB of CommonCrawl text across 100 languages — enormously more than mBERT's multilingual Wikipedia. CommonCrawl is web-scraped text, covering informal and formal language from across the internet.
- **RoBERTa training recipe applied:** No NSP task, dynamic masking, large batch sizes, longer training. All improvements from RoBERTa are applied in the multilingual setting.
- **Results:** Consistently and significantly outperforms mBERT on cross-lingual NER (WikiAnn), cross-lingual classification (XNLI), and multilingual QA (MLQA) across all 100 languages.
- **Two sizes:** XLNet Base (270M parameters) and XLNet Large (560M parameters).
- **Recommendation:** For any multilingual NLP task, XLNet is the default starting point. It is more reliable, more accurate, and broader in coverage than

4.1 BERT Variants at a Glance

Model	Params	Removes NSP?	Masking / Pre-train Task	Key Strength
-------	--------	--------------	--------------------------	--------------

BERT-Base	110M	No (has NSP)	Static random token MLM 15%	General baseline encoder
RoBERTa	125M	Yes	Dynamic random token MLM	Best pure English encoder
ALBERT	12–235M	No (SOP instead)	Token MLM + cross-layer sharing	Smallest parameter count
DistilBERT	66M	Yes	Distilled from BERT (all 3 losses)	Fastest production inference
SpanBERT	110M	Yes	Span masking + SBO	Best for QA / span extraction
XLNet	110–340M	No NSP concept	Permutation Language Modeling	No [MASK] mismatch, long docs
ELECTRA-Base	110M	Yes	Replaced Token Detection (RTD)	Most compute-efficient pre-train
DeBERTa-v3	86M–1.5B	Yes	Disentangled attention + ELECTRA	Best benchmark accuracy
ERNIE 2.0	110M+	No (many tasks)	Entity + phrase + discourse tasks	Knowledge-rich NLP
XLNet-RoBERTa	270–560M	Yes	Dynamic MLM on 100 languages	Best multilingual encoder

4.2 Key Takeaways

The Most Important Things to Remember

1. Transformer has three families: Encoder-Only (BERT), Decoder-Only (GPT), Encoder-Decoder (T5). Each solves different problems.
2. Encoders are bidirectional — they read full context. Decoders are causal — they generate left to right.
3. BERT's two pre-training tasks: MLM (forces bidirectionality) + NSP (sentence relationships, later found to be weak).
4. Every BERT variant fixes one specific limitation — know WHAT problem each variant solves, not just its name.
5. RoBERTa = same architecture, better training. No NSP, dynamic masking, more data.
6. ALBERT = smaller through parameter sharing + factorized embeddings.
7. DistilBERT = production-ready through knowledge distillation. 40% smaller, 60% faster.
8. ELECTRA = most training-efficient because it gets gradients from 100% of tokens, not 15%.
9. DeBERTa = disentangles content and position in attention — currently strongest for fine-tuning.
10. Domain variants (BioBERT, FinBERT, SciBERT) = same BERT architecture, continued pre-training on domain text.